Fine-tuning AlphaGenome in native JAX/Haiku

Alan Murphy[1], Masayuki Nagai[1], Alejandro Buendia[2], Anshul Kundaje[2], Peter K. Koo[1]

[1]Cold Spring Harbor Labs (CSHL)
[2]Stanford University

Genomics x AI Blog, 25 February 2026. https://doi.org/10.5281/zenodo.20272904
Online version: https://genomicsxai.github.io/blogs/2026-003/

Summary

This post introduces alphagenome-ft, a lightweight Python package for fine-tuning AlphaGenome using native JAX/Haiku.

We highlight workflows for

- adding custom prediction heads
- differing fine-tuning strategies
- freezing/unfreezing parameters
- attribution approaches

Here we focus on general workflows applicable to genome-scale assays and custom heads. For fine-tuning the encoder for short sequences such as MPRA, see this post.

Code: AlphaGenome fine-tuning utilities

Motivation

AlphaGenome [1] is a foundation sequence-to-function model trained on genome-scale data. Its native JAX/Haiku implementation is powerful but can be cumbersome to modify for custom tasks (we learned this the hard way!). Researchers often want to:

- Train a new head on a novel assay
- Apply low-rank adapters for efficient backbone updates
- Fine-tune the full model progressively
- Freeze certain components for stability
- Perform attribution analyses to gain insight into learned **cis**-regulatory logic

To help with this, we developed alphagenome-ft which provides a lightweight wrapper that achieves all these asks without modifying the original AlphaGenome codebase.

> *Side note:* AlphaGenome is a deep learning model that predicts functional genomic signals (e.g., accessibility, transcription, binding) directly from DNA sequence. We call such models sequence-to-function (seq2func) models.

> *Side note 2:* JAX/Haiku [2, 3] are DeepMind's frameworks which are similar to using PyTorch but optimized for large-scale accelerator workloads.

But wait, why fine-tune AlphaGenome?

Foundation sequence models like AlphaGenome are trained on diverse genome-scale assays, allowing them to learn general regulatory sequence features. However, most research questions involve **specific cell types, assays, perturbations, or organisms** that differ from the original training distribution.

You should first check if the foundation model alphagenome has an ouput track that's the same/similar to your cell type of interest, that might be enough! Otherwise, fine-tuning on your cell type/assay of interest is an option.

Fine-tuning adapts the pretrained model to these new contexts while preserving the regulatory knowledge already encoded in the backbone.

Benefits of fine-tuning include:

- **Improved performance with limited data** - Leverages pretrained regulatory features instead of learning from scratch.
- **Stability and efficiency via frozen parameters** - Freezing the backbone while training a new head reduces overfitting, lowers compute cost (this is important AlphaGenome is a BIG model -450m parameters), and prevents catastrophic forgetting.
- **Parameter-efficient adaptation** - Methods such as adapters or partial unfreezing allow targeted updates without retraining the full model.
- **Faster experimentation cycles** - New assays or prediction targets can be incorporated with minimal engineering effort.
- **Preservation of biological priors** - Retains learned sequence motifs and regulatory grammar that remain relevant across assays and cell types.

In practice, many workflows begin by training a task-specific head with the backbone frozen, then progressively unfreezing components if additional capacity is needed.

Fine-tuning with shorter sequence windows

By default, AlphaGenome is trained on ~1 million base-pair (1 Mb) input sequences, allowing the model to capture long-range regulatory interactions. However, during fine-tuning you are not required to use the full 1 Mb context.

If your downstream task does not depend strongly on ultra-long-range interactions, you can fine-tune using shorter input windows (e.g., 32 kb). This reduces memory usage, increases batch size flexibility, and can substantially speed up training.

This is particularly useful when:

- The signal of interest is predominantly local (functional outputs like chromatin accessibility are)
- You are adapting to assays with shorter effective regulatory range
- You want faster experimentation cycles

Importantly, this is different from encoder-only fine-tuning used for very short sequences (e.g., ~200–300 bp MPRA constructs). In that setting, only the convolutional encoder is used, bypassing the transformer and decoder entirely. This is covered in another post [4].

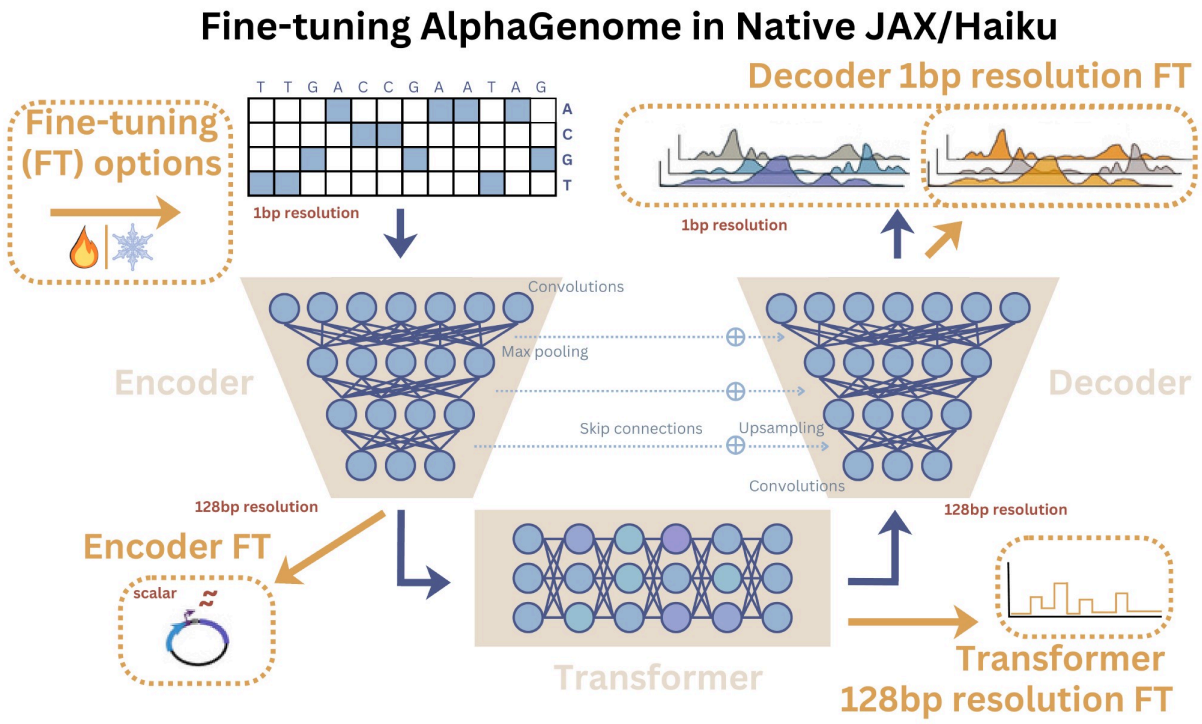
Here, we are still using the full model stack (encoder → transformer → decoder), but operating on a reduced genomic window.

In practice, reducing input length is a pragmatic trade-off between computational efficiency and long-range regulatory context (i.e. performance).

Package key features

- Custom prediction heads – easily register predefined, template, or fully custom heads
- Flexible parameter freezing – freeze backbone, individual modules, or heads
- Seamless integration – works with pretrained AlphaGenome weights
- Parameter inspection – explore and count model parameters
- Attribution analysis – gradient-based or in silico mutagenesis (ISM) methods
- Native JAX/Haiku – fully compatible with original AlphaGenome pipelines

AlphaGenome fine-tuning workflows schematic



Schematic of alphagenome-ft. alphagenome-ft enables fine-tuning of AlphaGenome (architecture shown) from different, modular stages of the model (the encoder - for short sequences, the transformer - for 128 base-pair resolution, and the decoder - 1 base-pair resolution). You can control what parts of the model are frozen or free to update and you can calculate attributions, all in native JAX/Haiku.

Usage

If these features don’t win you over, let’s walk through how easy it is to use:

Installation

alphagenome-ft wraps AlphaGenome and AlphaGenome Research and is available through pip . Installation requires three steps:

```
# Step 1: Install AlphaGenome and Research
pip install git+https://github.com/google-deepmind/alphagenome.git
pip install git+https://github.com/google-deepmind/alphagenome_research.git

# Step 2: Install alphagenome-ft
pip install alphagenome-ft
```

Python ≥ 3.11 is required. All other dependencies (JAX, Haiku, optax, etc.) are handled automatically.

Quick Start: Adding new heads

There are two main ways to add heads to AlphaGenome with the package (see the figure above for architecture references):

1. **Predefined heads**

Use existing AlphaGenome head types, e.g., rna_seq, atac, chip_tf:

```
from alphagenome_ft import (
    get_predefined_head_config,
    register_predefined_head,
    create_model_with_heads,
)

rna_config = get_predefined_head_config("rna_seq", num_tracks=4)
register_predefined_head("K562_rna_seq", rna_config)
model = create_model_with_heads("all_folds", heads=["K562_rna_seq"])
model.freeze_except_head("K562_rna_seq")

#Now ready to train!
```

2. **Custom heads and reference templates**

Our template heads give guidance on accessing different embeddings, which correspond to different biological resolutions - base-pair (bp) precision, regional regulatory context, and short-sequence feature extraction:

- StandardHead – 1bp embeddings (decoder output)
- TransformerHead – 128bp embeddings (transformer output)
- EncoderOnlyHead – CNN encoder output, <1 kb sequences (encoder output)

Note: Template heads are there as a guide for to how to set up your own custom head rather than a definitive ‘best’/‘standard’ option. You should update these with your own layer and loss function choices to fit your data needs.

```
from alphagenome_ft import templates, CustomHeadConfig, CustomHeadType, register_custom_head
```

```
register_custom_head(
    'my_head',
    templates.StandardHead,
    CustomHeadConfig(type=CustomHeadType.GENOME_TRACKS,
                     output_type='rna_seq',
                     num_tracks=1)
)
```

Workflows

A full selection of four workflows are given in our github repository, covering:

- Heads-only fine-tuning (frozen backbone)
- LoRA-style adapters (parameter-efficient fine-tuning)
- Full-model fine-tuning
- Encoder-only (MPRA / short sequences)

See the dedicated MPRA post [4] for full post dedicated to Encoder-only fine-tuning (it's great, even though I may be slightly biased as the one who wrote it …).

> *Side note:* Low-rank adapters (LoRA) [5] enable parameter-efficient fine-tuning by learning small update matrices instead of modifying the full backbone.

If unsure where to start, we recommend training a task-specific head with the backbone frozen, then progressively unfreezing components if additional capacity is needed.

Some extra functionality you might be interested in:

Parameter management and checkpoints

alphagenome-ft allows:

- Modular freezing: encoder, transformer, decoder
- Freezing all heads except one: `model.freeze_except_head('my_head')`
- Saving checkpoints (heads-only or full model)
- Loading with custom head registration

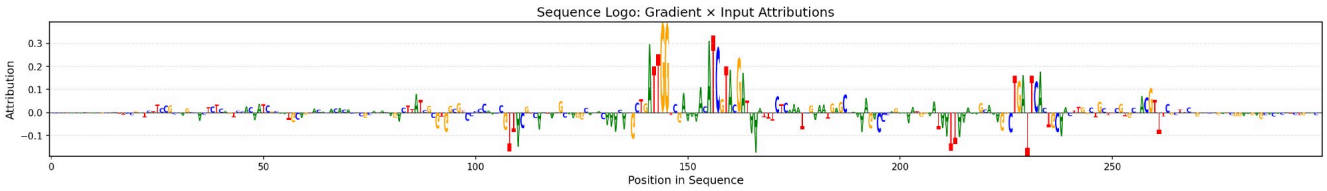
Attribution analysis

After training, alphagenome-ft also supports:

- DeepSHAP-like attributions [6, 7] - using dinucleotide shuffled reference sequences
- Gradient × Input [8]
- Gradient [9]
- In silico mutagenesis (ISM) [10, 11]

> *Side note:* Attribution methods highlight which nucleotides drive predictions in these models, helping reveal regulatory motifs and sequence grammar learned by the model.

You can also visualise contributions with `plot_attribution_map` or `plot_sequence_logo` functions! See below for an example attribution map when we fine-tuned AlphaGenome's encoder on fly STARR-seq data [12] - See our MPRA post [4] for more details.



Gradient x Input attribution map for AlphaGenome encoder only fine-tuning on fly STARR-seq data - See our MPRA post for more details on this fine-tuning. We can see the model highlights an AP-1 motif around position ~230, consistent with known enhancer regulatory logic.

You can see we recover the AP-1 motif (`TGAsTCA`) comes up at roughly position 230 which is a known regulator for developmental genes in flies [12].

Implications

To take a step back, what do we get with alphagenome-ft? AlphaGenome becomes flexible to:

- Rapid adaptation to new tasks
- Modular freezing/unfreezing for stability
- Supports genome-scale or perturbation assays
- Enables downstream interpretability

AlphaGenome can now be adapted as easily as modern vision and language foundation models — opening the door to rapid regulatory genomics experimentation. So if you think AlphaGenome could be useful if applied to your research, take a look at our package!

> alphagenome-ft brings foundation-model-style transfer learning workflows to regulatory genomics.

Compute requirements

A full analysis of the runtime requirements for fine-tuning AlphaGenome is highlighted in separate blog post [13] but in short:

- Fine-tuning with a frozen backbone, i.e. head-only fine-tuning will fit on a single, middle of the range GPU, maxing out at around 14-27 GB vram (batch size 1).
- For full Fine-tuning, you will need at least 76.1 GB vram, so a H100/H200 GPU.

Code and tutorials

- Source code & utilities
- Colab notebooks: Encoder Fine-tuning (MPRA) | Heads-only Fine-tuning
- Benchmarking AlphaGenome on NVIDIA GPUs: latency, memory, and feasibility across sequence lengths

TL;DR

- **AlphaGenome** is a powerful sequence-to-function foundation model, but adapting it natively in JAX/Haiku can be cumbersome.
- **alphagenome-ft** provides a lightweight wrapper for:
  - adding custom or predefined prediction heads
  - freezing and unfreezing specific modules
  - parameter-efficient fine-tuning (e.g., adapters)
  - running attribution analyses
- Most workflows can start by training a task-specific head with the backbone frozen, then progressively unfreezing if needed.
- This enables rapid adaptation to new assays while preserving pretrained regulatory knowledge.

If you want to fine-tune AlphaGenome without modifying its core codebase, alphagenome-ft is designed to make that process modular, efficient, and reproducible!

References

1. Avsec, Ž., Latysheva, N., Cheng, J., Novati, G. et al. Advancing regulatory variant effect prediction with AlphaGenome. *Nature* 649, 1206–1218 (2026). https://doi.org/10.1038/s41586-025-10014-0
2. Bradbury, J. et al. JAX: composable transformations of Python+NumPy programs (2018). https://github.com/jax-ml/jax
3. Hennigan, T., Cai, T., Norman, T., Martens, L. & Babuschkin, I. Haiku: Sonnet for JAX (2020). https://github.com/google-deepmind/dm-haiku
4. Murphy, A., Durán, A. & Koo, P. K. Adapting AlphaGenome to MPRA data. *Genomics x AI Blog* (2026). https://doi.org/10.5281/zenodo.20272900
5. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. & Chen, W. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv* 2106.09685 (2021). https://arxiv.org/abs/2106.09685
6. Lundberg, S. M. & Lee, S.-I. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems* 30 (2017). https://arxiv.org/abs/1705.07874
7. Shrikumar, A., Greenside, P. & Kundaje, A. Learning important features through propagating activation differences. *Proceedings of the 34th International Conference on Machine Learning*, PMLR 70, 3145–3153 (2017). https://arxiv.org/abs/1704.02685
8. Shrikumar, A., Greenside, P., Shcherbina, A. & Kundaje, A. Not just a black box: learning important features through propagating activation differences. *arXiv* 1605.01713 (2016). https://arxiv.org/abs/1605.01713
9. Simonyan, K., Vedaldi, A. & Zisserman, A. Deep inside convolutional networks: visualising image classification models and saliency maps. *arXiv* 1312.6034 (2013). https://arxiv.org/abs/1312.6034
10. Alipanahi, B., Delong, A., Weirauch, M. T. & Frey, B. J. Predicting the sequence specificities of DNA- and RNA-binding proteins by deep learning. *Nat. Biotechnol.* 33, 831–838 (2015). https://doi.org/10.1038/nbt.3300
11. Zhou, J. & Troyanskaya, O. G. Predicting effects of noncoding variants with deep learning–based sequence model. *Nat. Methods* 12, 931–934 (2015). https://doi.org/10.1038/nmeth.3547

12. de Almeida, B. P., Reiter, F., Pagani, M. & Stark, A. DeepSTARR predicts enhancer activity from DNA sequence and enables the de novo design of synthetic enhancers. *Nat. Genet.* 54, 613–624 (2022). https://doi.org/10.1038/s41588-022-01048-5
13. Tu, X., Buendia, A., Kundaje, A. & Mostafavi, S. Benchmarking AlphaGenome on NVIDIA GPUs: latency, memory, and feasibility across sequence lengths. *Genomics x AI Blog* (2026). https://doi.org/10.5281/zenodo.20272910